

Wprowadzenie do programowania w R

— cz. 2

Pakiety

Kod biblioteczny R jest zorganizowany w pakiety. Obiekty z pakietu są osiągalne w bieżącym środowisku po wywołaniu funkcji `library()`, np.

```
library(MASS)
```

Dołączać można tylko pakiety zainstalowane. Podstawowy sposób instalacji polega na wywołaniu funkcji `install.packages()`, np.

```
install.packages("plot3D")
```

Listy (wektory uogólnione)

Mogą zawierać obiekty dowolnego typu.

```
l <- list(a = c(2, 3, 4), f = function(x) sum(x^2), b = "Ala ma kota")
l
```

Do indeksowania list można używać operatora `[]`, ale ponieważ zwraca on podwektor tego samego typu, co obiekt, na którym jest wywołany (czyli w tym przypadku *podlistę*), nie daje bezpośredniego dostępu do elementów listy, np.

```
l[1]
l[1][2]
l[2]
l[2](3)
```

Natomiast operator `[[[]]` taki dostęp daje:

```
l[[1]]
l[[1]][2]
l[[2]]
l[[2]](3)
```

Jeśli elementy listy mają nazwy, to można tych nazw używać jako indeksów. Dla list z nazwami dostępny jest też specjalny operator indeksowania `$`.

```
l[['a']]
l$a
l[['a']][2]
l$a[2]
l$f(3)
```

Listy używane są powszechnie do zwracania złożonych wartości z funkcji.

```
f <- function() {  
  list(x = 10, y = 32)  
}  
  
f()$x
```

Czynniki (typy kategoriyczne, wyliczeniowe)

Czynnik jest typem o skończonym zestawie wartości, a więc reprezentuje zmienne kategorične (uporządkowane lub nie). Wewnętrznie reprezentowany jest przez wektor całkowitoliczbowy. Czynniki powstają z wektorów, których składowe przyjmują wartości z pewnego (zwykle niewielkiego) zbioru. Są specjalnie traktowane przez wiele funkcji przetwarzających dane.

```
directions <- factor(c("N", "E", "S", "W", "E", "E", "N", "S"))  
directions  
table(directions)  
pressures <- factor(c("High", "High", "Low", "Medium", "Low", "Low"), levels = c("Low", "Medium", "High"), ordered = TRUE)  
pressures  
table(pressures)  
x <- sample(1:10, 100, replace = TRUE)  
y <- sample(1:1000/1000, 100, replace = TRUE)  
plot(x, y)  
fx <- factor(x)  
plot(fx, y)
```

Programowanie obiektowe — poziom S3

W R dostępne (i używane) są co najmniej 3 różne podejścia do programowania obiektowego. Najprostszym i najbardziej elastycznym z nich jest tzw. poziom S3. Klasę obiektu na tym poziomie wyznacza wartość właściwości `class()`.

```
x <- 1  
class(x)  
class(x) <- "super_fighter"  
class(x)
```

Do sprawdzania przynależności obiektu do klasy służy funkcja `inherits()`.

```
inherits(x, "super_fighter")  
inherits(x, "numeric")  
inherits(2, "numeric")
```

Obiekt może mieć kilka klas.

```
class(x) <- c("Ala", "Ola", "Ula")  
class(x)  
inherits(x, "Ala")  
inherits(x, c("Ola", "Ela", "Ala"))  
inherits(x, c("Ola", "Ela", "Ala"), which = TRUE)
```

Drugim filarem obiektowości na poziomie S3 są **funkcje generyczne**. Funkcją generyczną jest funkcja o specjalnej definicji:

```
fff <- function(x, y) {  
  UseMethod("fff")  
}
```

Dla funkcji generycznej definiuje się jedną lub więcej **metod** dla poszczególnych klas. Nazwa metody jest konstruowana według schematu `funkcja_generyczna.typ , np.`

```
fff.numeric <- function(x, y) {  
  x + pi * as.numeric(y)  
}  
  
fff.super_fighter <- function(x, y) {  
  "I am a super fighter!"  
}
```

Podczas wywołania właściwa metoda dobierana jest na podstawie klasy pierwszego argumentu, czyli np. wywołanie funkcji generycznej `fff()` na obiekcie klasy `super_fighter` skutkuje wywołaniem metody `fff.super_fighter()`.

```
z <- 1  
fff(z, 13)  
class(z) <- "super_fighter"  
fff(z, 13)
```

Jeśli obiekt należy do kilku klas, wybiera się tę z metod, której klasa występuje najwcześniej w wektorze klas obiektu

```
class(z) <- c("numeric", "super_fighter", "alamakota")  
fff(z, 13)
```

Jeśli dla żadnej z klas obiektu, na którym wywoływana jest funkcja generyczna, nie jest zdefiniowana odpowiednia metoda, używana jest **metoda domyślna** o nazwie `funkcja_generyczna.default()`. Jeśli ta ostatnia nie jest zdefiniowana, zgłaszany jest błąd.

```
fff("ala", 13) #Ta linia zgłasza błąd!  
  
fff.default <- function(x, y) {  
  y  
}  
  
fff("ala", 13)
```

Biblioteka standardowa zawiera znaczną liczbę funkcji generycznych. Przykładami są: `print()`, `plot()`, `summary()`, `predict()` i cały zestaw operatorów. Z tego, co zostało napisane powyżej, wynika, że możemy zdefiniować własne metody dla tych funkcji.

```
print.super_fighter <- function(x) {  
  print(paste("Super fighter", x))  
}  
  
z <- 1  
class(z) <- "super_fighter"  
z
```

Rozkłady prawdopodobieństwa

Na ogół rozkłady reprezentowane są przez 4 funkcje:

- dxx - gęstość lub funkcja prawdopodobieństwa
- pxx - dystrybucja
- qxx - funkcja kwantylowa
- rxx - losowanie zgodne z rozkładem.

xx jest ciągiem znaków oznaczającym rozkład, np. unif - rozkład jednostajny, norm - normalny, binom - dwumianowy, pois - Poissona, t - t (Studenta), f - F (Fishera-Snedecora).

```
dnorm(2.3)  
pnorm(2.3)  
x1 <- rnorm(10)  
mean(x1)  
var(x1)  
sd(x1)  
x2 <- rnorm(10, mean = 1, sd = 5)  
mean(x2)  
var(x2)  
sd(x2)  
dpois(2, lambda = 1)  
rpois(10, lambda = 1)  
qpois(0.75, lambda = 1)
```

Dla osiągnięcia potwarzalności obliczeń z czynnikiem losowym stosuje się funkcję `set.seed()`

```
rnorm(10)  
rnorm(10)  
set.seed(2020)  
rnorm(10)  
set.seed(2020)  
rnorm(10)
```

Rysowanie — dane trójwymiarowe

Wykres konturowy (poziomicowy)

```
x <- seq(-pi, pi, length.out = 100)
y <- x
f <- outer(x, y, function(x, y) cos(y) / (1 + x^2))
contour(x, y, f)
contour(x, y, f, nlevels = 45)
fa <- (f - t(f)) / 2
contour(x, y, fa, nlevels = 15)
```

Mapa temperatury (*heatmap*, mapa hipsometryczna)

```
image(x, y, f)
image(x, y, f, col = topo.colors(32, alpha = 1))
contour(x, y, f, add = TRUE)
```

Wykres trójwymiarowy w perspektywie

```
persp(x, y, f)
persp(x, y, f, theta = 30)
persp(x, y, f, theta = 30, phi = 40)
persp(x, y, f, theta = 30, phi = 40, shade = 0.75, border = NA)
```

[Ładniejsze wykresy trójwymiarowe można wykonać korzystając z funkcji z pakietu `plot3D`.]

Ramki danych i dane zewnętrzne

Ramka danych jest szczególnym rodzajem listy składającej się z wektorów o równej długości.

```
df <- data.frame(nazwa = c("Tatooine", "Dagobah", "Hoth"),
                 srednica = c(20, 37.4, 11.22),
                 klimat = factor(c("pustynia", "bagno", "śnieg")))
df
```

Tworzy to macierzopodobną strukturę dwuwymiarową, stanowiącą podstawowy sposób reprezentacji danych obserwacyjnych w R. W standardowej interpretacji kolumny (powstałe z wektorów wejściowych) reprezentują obserwowane *zmienne*.

```
df$srednica
df[, 2]
df["srednica"]
df[2]
```

Warto zwrócić uwagę, że dwie pierwsze instrukcje produkują wektor, a dwie ostatnie jednokolumnową ramkę danych. Natomiast wiersze ("przekroje poziome") reprezentują poszczególne *obserwacje*.

```
df[2, ]
```

Wczytywanie danych zewnętrznych

```
Auto <- read.table("Auto.data")
class(Auto)
Auto
```

Dane są wczytywane do ramki danych. Do zapisywania ramek danych do pliku tekstowego służy funkcja `write.table()` .

W powyższym przypadku dane zostały wczytane błędnie, bo plik zawiera wiersz nagłówka i dane nieznane (oznaczone `?`).

```
Auto <- read.table("Auto.data", header = TRUE, na.strings = "?")
names(Auto)
head(Auto)
```

Wczytywanie z pliku CSV

```
Auto <- read.csv("Auto.csv", header = TRUE, na.strings = "?")
head(Auto)
```

W przypadku danych w formacie właściwym dla polskiej lokalizacji bardziej odpowiednia jest funkcja `read.csv2()` .

Usuwanie wierszy z wartościami nieistniejącymi

```
dim(Auto)
Auto <- na.omit(Auto)
dim(Auto)
```

Indeksowanie ramki danych.

```
Auto[2, 5]
Auto[2, "weight"]
Auto[, 5]
Auto[, "weight"]
Auto[1:4]
Auto[1:4,]
Auto[3:10, c(2, 5)]
Auto[3:10, c("cylinders", "weight")]
```

Indeksowanie charakterystyczne dla list.

```
Auto$mpg
Auto$weight[1:4]
Auto$cylinders[Auto$mpg > 30]
Auto[Auto$mpg > 30,]
attach(Auto)
mpg
mpg[mpg > 30]
cylinders[mpg > 30]
Auto[mpg > 30,]
Auto[mpg > 30, "weight"]
```

Prezentacja graficzna danych z ramki

```
plot(cylinders, mpg)
plot(as.factor(cylinders), mpg)
hist(mpg)
hist(mpg, col = "red")
hist(mpg, col = "red", breaks = 15)
pairs(Auto)
pairs(~ mpg + displacement + horsepower + weight + acceleration, data = Auto)
```

Identyfikacja wartości interesujących zmiennych

```
plot(horsepower, mpg)
identify(horsepower, mpg, name)
```

Podsumowania statystyczne

```
summary(Auto)
summary(mpg)
```